

Project 3: Critters

1. Overview

This assignment asked us to write the Interpreter for Critter. The simulator was already given as a compiled JAR (with the grid, graphics, turn order, movement and eating rules, etc.), and we had to implement the brain, which reads the program `CritterSpecies` and executes the code for Critter on each tick. A behavior file describes each species with its name on the first line, followed by one action per line, and terminated by a blank line to ignore the comments afterward. The instruction set code is written in a new species programming language. We categorized all the possible commands into: the five actions (hop, left, right, eat, infect), unconditional and conditional jumps (go, ifempty, ifwall, ifenemy, ifally, ifangle, ifhungry, ifstarving, ifrandom, iflt, ifeq, ifgt), and register instructions (write, add, sub, inc, dec) that operate on ten registers (r1...r10).

We implemented two methods from the Interpreter interface. `loadSpecies` was called once for each behavior file at startup, translating the behavior file into an object of type `CritterSpecies`. `executeCritter`, which is called once per critter per turn, runs the Critter object's program from the point at which it was last paused until it has completed an action, then saves the next line. Because the simulator never asks the simulation to stop, the main requirement (along with the other correctness checks) was robustness, meaning we had to ensure a badly written critter or incorrectly formatted behavior file shouldn't crash the entire simulation.

2. Solution Design

First, we will explain `loadSpecies`. It begins by opening the file with `BufferedReader`. The first line reads the species name as a standalone line, and if the line is blank, we return null to indicate the file can't be parsed, as the interface requires. We then loop reading instruction lines until we hit a blank line or the end of file. If there is no blank line at the end of a file, `readLine()` will return null, and calling `isBlank()` on null throws a `NullPointerException`. The null check is first in the loop condition to avoid this. We then trim each instruction before storing it, and if the name line was non-blank but there are no instructions, we return null. Anything after the blank line is never read, so comments at the bottom of a behavior file are ignored. We throw any `IOException` out of the method, since the interface states that it can be thrown for files it can't open or read.

Now, we will explain the `executeCritter`. Recall that from `loadSpecies`, the list of instruction strings contained in `codeList` and the species name were stored separately (not in the list). The numbering in behavior files starts at 1 (and `getNextCodeLine()` returns 1-indexed line numbers), although Java lists are 0-indexed. At the beginning of each call to `executeCritter`, we read the critter's next code line using `getNextCodeLine` before the loop. Inside the loop, we split the current instruction into a string list with `trim().split("\\s+")`, so any extra whitespace or tab characters between arguments are removed. We then use a long chain of if/else statements to compare the first element of the string array, which should contain the instruction. Each instruction falls into one of three categories: actions, jumps, and registers. Actions call the corresponding Critter method, increment the next code line, and return, ending the turn. Everything else only modifies the local variable `currentLine`. For example, for the line jump actions, we have to ensure jumps set the next line to the target line using the `lineJump` helper method to essentially translate the register or increment/decrement to the desired line. These include `go`, `ifhungry`, `ifstarving`, `ifrandom`, `ifangle`, `ifwall`, `ifenemy`, and `ifally`. For register actions, such as `iflt`, `ifeq`, `ifgt`, `write`, `add`, `sub`, `inc`, and `dec`, we parse the register index and either update its value directly with `setReg` or conditionally update `currentLine`. The register instructions and conditions that fail skip to the next line, and the loop begins again on the next turn.

Now, we will explain our `lineJump` helper method in greater detail. The spec says every jump instruction can have three notations for its target, so we have that logic in a single helper that all 12 jumping instructions call. If the target starts with an `r`, it is a register, and we jump to the line number stored in that register. If it starts with a `+` or `-`, it is relative, and `Integer.parseInt` preserves the sign so we can just add it to the current line to get our target. Anything else is just an absolute line number.

Now, we will explain the `parseReg` helper method. The register arguments, such as the `r1` in `add r1 r2`, require the register's number, not just its value. However, we only built `lineJump` to return the value held in the register. So for those register arguments, we implemented a separate `parseReg` helper that verified the token started with `r`, then read the number following, then checked the number was `>0` and `<=Critter.REGISTERS` and returned the number. `LineJump` reused `parseReg` for its register case, ensuring range checking happened consistently.

Now, for error handling. We ensured every helper throws an `IllegalArgumentException` with an informative message. The entire body of the execution loop is inside a try/catch catching `IllegalArgumentException` and `IndexOutOfBoundsException`. Any instruction that is not recognized falls into a final else that throws an `IllegalArgumentException` too. Our catch block prints the line number and the message to standard error, then returns. This means a critter that goes to a bad instruction does nothing on that turn and on subsequent turns. As for out-of-bounds lines, before executing each instruction, we test whether `currentLine` is in `[1..instructions]`. If not, we return without executing anything.

3. Discussion

All of the required instructions have been implemented, including the optional argument to infect, all three jump target notations for each jumping instruction, and ifRandom through the Critter interface's own ifRandom() method rather than our own random number generator. All of the behavior files provided, including Flytrap, Food, Hop, and Rover, loaded properly.

The major flaw in our design is that validation occurs at runtime rather than load time. LoadSpecies only looks at the text of a file and does not verify that each line is a valid instruction. Even though the API specifies that loadSpecies should return null if a file cannot be loaded, we allow an invalid instruction to be placed in the file, as the only indication of failure occurs when a critter gets to that line in the text, at which point the error message is printed on every turn for all critters of the same species, since the critter remains in the same location on the error line.

Our solution was to place all our validation into the executeCritter function, and never crash the simulator. Moving validation into loadSpecies would be the lowest-hanging fruit to improve this design.

A few other minor limitations also exist in our code. Bearings are not validated, which means an instruction such as ifEmpty 30 4 will not generate an error when it should. The second argument of ifAngle and the line number passed to infect are also passed through without any range validation. Our last limitation is that because we re-split each instruction string every time it executes, our interpreter re-computes some parsing work on each turn, which is not as noticeable or relevant here but would definitely be the first thing to optimize when working on a larger scale.

4. Testing Methodology

The project asks why the simulator itself is so bad for testing, and after hands-on experience, we can identify various reasons. Firstly, it is nondeterministic, meaning the way critters are placed, the order in which they go, and ifRandom all rely on complete randomness, and so the same program can lead to very different runs. Second, we have no way of fixing the state of the testing world around a given critter, so it can be hard to get a certain combination of circumstances, like “enemy at bearing 90 while starving,” to occur. Third, we can only watch by staring at the display, not by looking at the saved line number and register values, so a bug that causes the output to jump to another line looks like a fix.

What we did instead was to construct a test harness in JUnit with a Test Critter class which implements the Critter interface. It allows the test to set the program directly, set the contents of each of the 8 surrounding cells and the off-angle at each bearing, set the hunger value, and read/write registers. It doesn't do anything in its action methods, but instead notes the last action

it took and which line to call infect with. This turns each test into a controlled experiment, where we set all of the inputs, call `executeCritter` once, and then verify both the chosen action and the saved next line.

Our 18 tests are organized around the main parts of the interpreter. For loading species, we test loading the provided Rover file, check that `loadSpecies` stops at the blank line and ignores any comments after it, and check that it trims extra whitespace around the name and instructions. The last test uses a file with no trailing blank line, so it also checks how we handle the end of input files. In terms of actions, we test hop, eat, and infect both with and without its optional argument. We check each time that the next line is saved correctly. We also run left followed by right across two consecutive calls, which confirms that a critter resumes where it left off between turns rather than starting off somewhere else or changing positions. For jumps, we test go with an absolute target, a relative “+” target that jumps forwards, a relative “-” target that jumps backwards, and a register target. For registers, we test write, inc, and dec, as well as add and sub, including a subtraction that produces a negative value. For our conditions, we test `iflt` and `ifeq` when the condition is true, `ifgt` when it is false, and `ifempty` in both the empty case and the enemy-ahead case.

The harness also validated itself before we finished the interpreter. When we initially ran the tests on the interpreter, we only passed 1 of our 18 test cases. It wasn't an instruction logic mistake, but rather an issue about how we created our `codeList` from the inputted behavior code. In our first try, our code stored the species name at index 0 in the code list, which meant that all our programs were off by one line.

When we saw almost every single test failing, we realized it was always by exactly one line. Seeing this showed where our assumption about the data structure was wrong. Once we changed the interpreter so that we stored the name separately, all 18 of the tests passed.

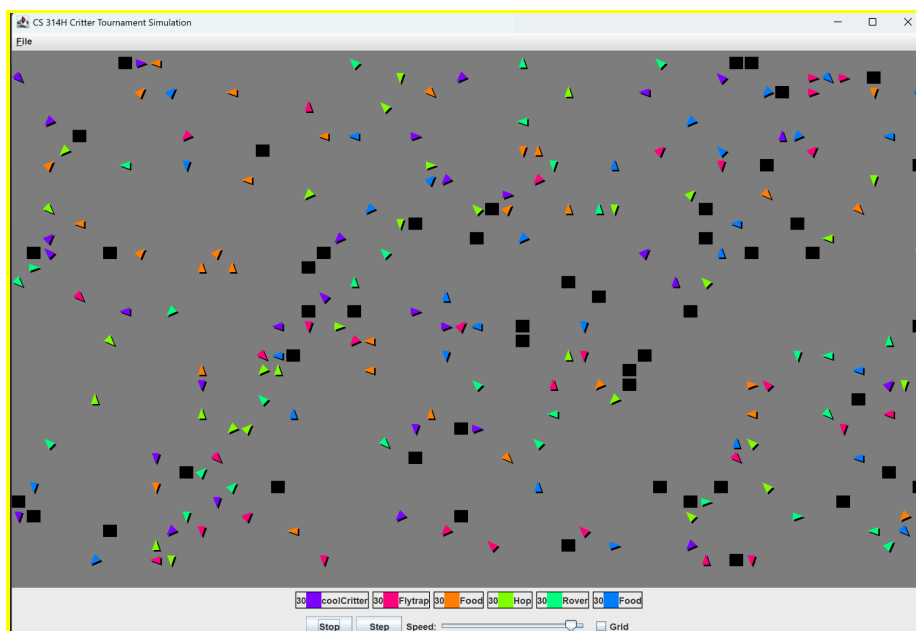
We also used the test harness to discover the infinite loop bug when we loaded a species with only one instruction of “go 1”. When we did this, the critter would go on forever, which led us to implement a step guard to end the game. However, we then remembered that we are not required to check for this in our interpreter, so we removed that function from the code. Since `TestCriticter.ifRandom()` returns a random value, we can't test the `ifRandom` function yet, unless we give the harness a way to force the result of the random call.

5. Our Critter

Our critter, `coolCriticter`, is saved as `chauhan.cri`. `coolCriticter` is built on the same core concepts as the provided Rover critter, essentially just moving forward and then attacking whatever is in front of it. However, it also actively turns to face enemies instead of waiting for them to wander right in front of it. On every turn, it runs through a fixed list of checks in a prioritized order. First,

if an enemy is directly ahead, it attacks. It infects the enemy by default, which converts it into another coolCriticter and grows our population, but if coolCriticter is hungry or starving, it eats the enemy instead. Eating instead of infecting the critter resets its hunger and prevents the critter from slowing down, so this choice keeps the population from slowing down or starving while still having an inclination to grow whenever possible. Second, if an enemy is directly to its right or left, it turns towards that enemy. Then, on the next turn, the enemy is in front of it, and the first check applies. Third, if the square ahead is empty, it hops forward. Finally, if the path is blocked by a wall or an ally, it turns left or right at random using ifrandom, which prevents a group of coolCriticters from getting stuck while facing the same wall. Every action is followed by go 1, so coolCriticter re-scans its surroundings from the beginning of its behavior code on every turn instead of continuing a fixed sequence of moves.

We tested coolCriticter against each of the provided critters in a battle arena with 30 critters of each species. We conducted 10 trials total, and coolCriticter won every trial finishing with an average population of 13 while eliminating the other species entirely. Rover was the closest to matching up with coolCriticter, and was the last species alive aside from coolCriticter before dying. This makes sense, since Rover also infects anything in front of it, and two critters that both attack on sight largely trade members back and forth. coolCriticter's advantage of turning toward side enemies was enough to give it the edge more often than not, but not decisively.



coolCriticter does have some limitations. It only looks directly ahead and directly to its sides, so an enemy at a diagonal or behind it goes unnoticed until one of them moves. It also has no memory and doesn't use registers, so it cannot remember where it last saw an enemy or coordinate with

nearby allies. These changes could optimize coolCritic's performance to perform better, but would be much more complex to implement. A stronger version would be able to check all eight bearings and use a register to count turns spent without eating.

6. Pair Programming

Log:

Sep 21st: 8PM - 11PM(Prithviraj and Reva) Understanding Project and Interpreter. Reva drove for 3 hours.

Sep 22nd: 8PM - 12:45AM(Prithviraj and Reva) Interpreter. Prithvi drove for 3 hours, Reva drove for 2 hours.

Sep 23rd: 3PM-4:30PM and 9PM - 2AM(Prithviraj and Reva) Interpreter, Test Cases, and Report. Prithvi drove for 1.5 hours, Reva drove for 3 hours, Prithvi drove for the last 2 hours.

Sep24th: 6PM-7PM(Reva) Test Cases

Sep 25th: 8PM - 11PM(Prithviraj and Reva) Report

Pair programming had various pros and cons for our partnership. Working together meant we could catch each other's mistakes when it came to logic for the code. When one of us was driving, the other could spot problems as they were being written, such as a jump that would be overwritten on the next loop iteration or a line number that was off by one, instead of finding them hours later in the simulator. Pairing also meant we could explain things the other person didn't understand. When one of us was confused about how a part of the interpreter fit into the whole, such as how the saved line number carries a critter's place from one turn to the next, the other could walk through it right away, and we both came out with a better understanding of the full design than either of us would have had alone. Finally, there was less pressure to get everything perfectly correct on the first try. Knowing that someone else would also be thinking about edge cases and reviewing the code meant each of us had guardrails, which made it easier to write a first version, test it, and improve it rather than getting stuck trying to make it flawless before moving on.

Pair programming also had real costs. Implementation often took longer because before writing anything we had to discuss and agree on an approach, and some decisions, like how errors should be handled or how lines should be indexed, took several conversations to settle. Communication around GitHub was another challenge. Since we sometimes worked on the same files separately, we had to make sure we pushed our changes and pulled the other person's before starting, and when we forgot, we ran into merge conflicts that had to be resolved before we could keep working. Lastly, finding times when we were both available was difficult with our different class

and work schedules, and unfortunately many of our sessions ended up happening late at night, when it was harder to stay focused and think carefully about the code.

7. Social Impact

CritterFest gives us an actual place for our respective programs to compete, and this is very fruitful as it pushes people to be more creative, since a critter can only win if its author thinks carefully about strategy, edge cases, and how the other critters will behave. This is very applicable to coding in real life, where competitions between companies and benchmarks drive rapid progress in computer science. A real-world example of this is seen right now with top AI labs in the US competing to see who can reach AGI first. At the same time, competition can reward winning at any cost, and in contexts like gaming it can often be associated with toxic behavior towards other players and competition. This class contest will work best when it is treated as a way to learn from other people's designs rather than only as a ranking. This was also our first pair-programmed assignment, and working in a pair is much more realistic, as it's how software is built in industry and research. Pairing people forces you to explain your reasoning out loud, which can help catch mistakes earlier, but can also mean that design decisions like how lines are numbered have to be agreed on explicitly rather than assumed, because two people can each make a reasonable assumption that does not match the other's.